1.Creating Arrays and Array Operations:

a)Create a 1D array with elements from 0 to 9.

b)Create a 2D array with shape (3, 4) filled with random numbers.

c)Calculate the mean and standard deviation of a given array.

d)Normalize the values of an array (subtract the mean and divide by the standard deviation).

```
import numpy as np
# a) Create a 1D array with elements from 0 to 9
arr1 = np.arange(10)
print("1D Array:")
print(arr1)
# b) Create a 2D array with shape (3,4) filled with random numbers
arr2 = np.random.rand(3, 4)
print("\n2D Random Array:")
print(arr2)
# c) User input array
n = int(input("\nEnter the number of elements: "))
print("Enter the elements:")
arr = np.array([float(input()) for i in range(n)])
print("\nArray:", arr)
mean = np.mean(arr)
std = np.std(arr)
print("Mean =", mean)
print("Standard Deviation =", std)
# d) Normalize the array
normalized = (arr - mean) / std
print("Normalized Array:")
print(normalized)
```

2 Generating Array, Reshaping and Stacking:

a) Create a 1D array of 10 evenly spaced values between 0 and 1.

b) Generate a 3x3 identity matrix.

c) Reshape a 1D array into a 2D array with shape (2, 5).

d) Stack two arrays vertically and horizontally.

```
import numpy as np
# a) Create a 1D array of 10 evenly spaced values between 0 and 1
arr = np.linspace(0, 1, 10)
print("Evenly Spaced Array:")
print(arr)
# b) Generate a 3x3 identity matrix
identity = np.eye(3)
print("\nIdentity Matrix:")
print(identity)
# c) User input for reshaping
print("\nEnter 10 elements:")
array = np.array([int(input()) for i in range(10)])
reshaped = array.reshape(2, 5)
print("\nReshaped Array (2x5):")
print(reshaped)
# d) User input for stacking
print("\nEnter 4 elements for First 2x2 Matrix:")
a = np.array([int(input()) for i in range(4)]).reshape(2, 2)
print("\nEnter 4 elements for Second 2x2 Matrix:")
b = np.array([int(input()) for i in range(4)]).reshape(2, 2)
print("\nFirst Matrix:")
print(a)
print("\nSecond Matrix:")
print(b)
vertical = np.vstack((a, b))
```

```
horizontal = np.hstack((a, b))
print("\nVertical Stack:")
print(vertical)
print("\nHorizontal Stack:")
print(horizontal)
```

3 Indexing and Slicing & Matrix Operations:
a) Extract the third column from a 2D array.
b) Reverse the order of elements in a 1D array.
c) Create two matrices (2x3 and 3x4) and perform matrix multiplication.
d) Find the determinant of a 3x3 matrix.

```
import numpy as np
# a) Third column of a 2D array
a = np.array([[1,2,3,4],
              [5,6,7,8],
              [9,10,11,12]])
print("Third Column:", a[:,2])
# b) Reverse a 1D array
arr = np.array(list(map(int, input("Enter numbers: ").split())))
print("Reversed:", arr[::-1])
# c) Matrix multiplication
A = np.array([[1,2,3],[4,5,6]])
B = np.array([[7,8,9,10],
              [11,12,13,14],
              [15,16,17,18]])
print("Matrix Multiplication:\n", A @ B)
# d) Determinant of a 3x3 matrix
M = np.array(list(map(int, input("Enter 9 elements: ").split()))).reshape(3,3)
print("Determinant:", np.linalg.det(M))
```

4 Statistical Operations, Broadcasting:

a) Generate a random 2D array and calculate the mean along each axis.

b) Find the minimum and maximum values in a given array.

c) Create a 1D array and add a constant value to each element without using a loop.

d) Multiply each row of a 2D array by a different constant.

```
import numpy as np
# a) Mean of rows and columns
arr = np.random.randint(1, 100, (3, 4))
print("Array:\n", arr)
print("Row Mean:", np.mean(arr, axis=1))
print("Column Mean:", np.mean(arr, axis=0))
# b) Minimum and Maximum
a = np.array(list(map(int, input("Enter elements: ").split())))
print("Minimum:", np.min(a))
print("Maximum:", np.max(a))
# c) Add a constant to every element
c = int(input("Enter constant: "))
print("Result:", a + c)
# d) Row-wise multiplication
r, col = map(int, input("Enter rows and columns: ").split())
m = np.array([list(map(int, input().split())) for i in range(r)])
k = np.array(list(map(int, input("Enter row constants: ").split())))
print("Result:\n", m * k.reshape(r, 1))
```

5 Develop a python program to create pandas data frame from list of data.

```
# Import the pandas library
import pandas as pd
# Create a list containing student data
data = [
    ["John", 20, 85],
```

```
    ["Mary", 21, 92],
    ["Alex", 19, 78],
    ["David", 22, 88]
]
# Create a DataFrame from the list
df = pd.DataFrame(data, columns=["Name", "Age", "Marks"])
# Display the DataFrame
print("Student DataFrame")
print(df)
```

6. Develop a python program to analyze the dataset using pandas and matplotlib library

```
import pandas as pd
import matplotlib.pyplot as plt
# Create Dataset
data = {
    "Name": ["John", "Mary", "Alex", "David", "Sophia"],
    "Marks": [85, 92, 78, 88, 95],
    "Age": [20, 21, 19, 22, 20]
}
# Create DataFrame
df = pd.DataFrame(data)
# Display Dataset
print("Dataset")
print(df)
# Basic Analysis
print("\nAverage Marks:", df["Marks"].mean())
print("Highest Marks:", df["Marks"].max())
print("Lowest Marks:", df["Marks"].min())
# Plot Bar Graph
```

```
plt.bar(df["Name"], df["Marks"])
plt.title("Student Marks")
plt.xlabel("Student Name")
plt.ylabel("Marks")
plt.show()
```

7. Develop a program to compute Mean, Median, Mode, Variance and Standard Deviation using Datasets.

```
import numpy as np
# Dataset
data = np.array([12.5,12.5,12.5,12.5,12.5,
                 17.5,17.5,17.5,17.5,17.5,17.5,
                 22.5,22.5,22.5,22.5,22.5,22.5,22.5,22.5,22.5,
                 27.5,27.5,27.5,27.5,27.5,27.5,27.5,27.5,
                 32.5,32.5])
# Mean
mean = np.mean(data)
# Median
median = np.median(data)
# Mode
values, counts = np.unique(data, return_counts=True)
mode = values[np.argmax(counts)]
# Variance
variance = np.var(data)
# Standard Deviation
std = np.std(data)
# Display Results
print("Dataset:")
print(data)
print("\nMean =", mean)
print("Median =", median)
```

```
print("Mode =", mode)
print("Variance =", variance)
print("Standard Deviation =", std)
```

8.Plot histogram for the given dataset.

10-15	15-20	20-25	25-30	30-35
5	6	9	8	2

```
import numpy as np
import matplotlib.pyplot as plt
# Dataset
data = np.array([12.5,12.5,12.5,12.5,12.5,
                 17.5,17.5,17.5,17.5,17.5,17.5,
                 22.5,22.5,22.5,22.5,22.5,22.5,22.5,22.5,22.5,
                 27.5,27.5,27.5,27.5,27.5,27.5,27.5,27.5,
                 32.5,32.5])

# Plot Histogram
plt.hist(data,
         bins=[10,15,20,25,30,35],
         edgecolor='black')
plt.title("Histogram")
plt.xlabel("Class Interval")
plt.ylabel("Frequency")
plt.show()
```

9. Draw box-and-whisker plot for the data set {3, 7, 8, 5, 12, 14, 21, 13, 18}.

```
import matplotlib.pyplot as plt
# Dataset
data = [3, 7, 8, 5, 12, 14, 21, 13, 18]
```

```
# Draw Box-and-Whisker Plot
plt.boxplot(data)
# Add title and label
plt.title("Box-and-Whisker Plot")
plt.ylabel("Values")
# Display the plot
plt.show()
```

10. Draw Line Plot and Bar chart for the following data.

Elapsed time (s)	0	1	2	3	4	5	6
Speed(m/s)	0	3	7	12	20	30	45.6

```
import matplotlib.pyplot as plt
# Data
elapsed_time = [0, 1, 2, 3, 4, 5, 6]
speed = [0, 3, 7, 12, 20, 30, 45.6]
plt.figure()
plt.plot(elapsed_time, speed, marker='o')
plt.title("Elapsed Time vs Speed")
plt.xlabel("Elapsed Time (s)")
plt.ylabel("Speed (m/s)")
plt.grid(True)
plt.show()
plt.figure()
plt.bar(elapsed_time, speed)
plt.title("Elapsed Time vs Speed")
plt.xlabel("Elapsed Time (s)")
plt.ylabel("Speed (m/s)")
plt.show()
import numpy as np
```

11.Develop a python program to implement Simple linear regression and plot the graph

```
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
y = np.array([2, 4, 5, 4, 5])
model = LinearRegression()
model.fit(X, y)
plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear Regression")
plt.show()
```

12. Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.preprocessing import LabelEncoder
# Dataset
data = {
    "Outlook": ["Sunny", "Sunny", "Rainy", "Rainy", "Overcast"],
    "Temperature": ["Hot", "Mild", "Cool", "Mild", "Hot"],
    "Play": ["No", "No", "Yes", "Yes", "Yes"]
}
# Create DataFrame
df = pd.DataFrame(data)
# Convert text to numbers
```

```
le = LabelEncoder()
for col in df.columns:
    df[col] = le.fit_transform(df[col])
# Input and Output
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
# Create and train Decision Tree using Entropy (ID3)
model = DecisionTreeClassifier(criterion="entropy")
model.fit(X, y)
# New sample
sample = pd.DataFrame([[2, 1]], columns=["Outlook", "Temperature"])
# Prediction
prediction = model.predict(sample)
print("Prediction:", prediction)
# Display Decision Tree
plt.figure(figsize=(10, 6))
plot_tree(
    model,
    feature_names=["Outlook", "Temperature"],
    class_names=["No", "Yes"],
    filled=True,
    rounded=True
)
plt.title("Decision Tree using Entropy (ID3)")
plt.show()
```

13. Implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets.

Create a CSV file with file name naive_bayes_data.csv

```
Age,Income,Student,CreditRating,BuysComputer
```

Youth,High,No,Fair,No

Youth,Medium,Yes,Fair,Yes

Middle,High,No,Excellent,Yes

Senior,Low,Yes,Fair,Yes

Senior,Medium,No,Excellent,No

Middle,Low,Yes,Fair,Yes

Youth,Low,No,Excellent,No

Code - import pandas as pd

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import CategoricalNB
from sklearn.metrics import accuracy_score
# Read CSV
data = pd.read_csv("naive_bayes_data.csv")
# Input and output
X = data.iloc[:, :-1]
y = data.iloc[:, -1]
# Convert text data to numbers
for col in X.columns:
    X[col] = LabelEncoder().fit_transform(X[col])
y = LabelEncoder().fit_transform(y)
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
# Train model
model = CategoricalNB()
model.fit(X_train, y_train)
# Prediction
y_pred = model.predict(X_test)
```

```
# Accuracy
print("Actual:", y_test)
print("Predicted:", y_pred)
print("Accuracy:", accuracy_score(y_test, y_pred) * 100, "%")
```

14. Apply k means algorithm to cluster a set of data stored in a .CSV file.

Create a CSV file with file name kmeans_data.csv.

```
X,Y
1,2
2,3
3,4
8,9
9,10
10,11
import pandas as pd
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
# Read CSV
data = pd.read_csv("kmeans_data.csv")
# Select data
X = data[["X", "Y"]]
# Create K-Means model
model = KMeans(n_clusters=2, random_state=42)
# Train and predict clusters
data["Cluster"] = model.fit_predict(X)
# Print result
print(data)
# Plot clusters
plt.scatter(data["X"], data["Y"], c=data["Cluster"])
plt.xlabel("X")
```

```
plt.ylabel("Y")
plt.title("K-Means Clustering")
plt.show()
```

15. Implement k-Nearest Neighbor algorithm to classify the iris data set. Print both correct and wrong predictions.

iris.csv

```
SepalLength,SepalWidth,PetalLength,PetalWidth,Species
5.1,3.5,1.4,0.2,setosa
4.9,3.0,1.4,0.2,setosa
5.0,3.6,1.4,0.2,setosa
4.7,3.2,1.3,0.2,setosa
6.4,3.2,4.5,1.5,versicolor
6.9,3.1,4.9,1.5,versicolor
6.3,3.3,6.0,2.5,virginica
6.5,3.0,5.8,2.2,virginica
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
# Read Iris dataset from CSV
data = pd.read_csv("iris.csv")

# Input features
X = data.iloc[:, :-1]
# Output class
y = data.iloc[:, -1]
# Split data into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.25, random_state=42
)
# Create KNN model
model = KNeighborsClassifier(n_neighbors=3)
# Train the model
model.fit(X_train, y_train)
# Predict test data
y_pred = model.predict(X_test)
correct = 0
wrong = 0
# Print predictions
for actual, predicted in zip(y_test, y_pred):
    if actual == predicted:
        print("Correct Prediction:", actual, "->", predicted)
        correct += 1
    else:
        print("Wrong Prediction:", actual, "->", predicted)
        wrong += 1
# Print results
print("\nCorrect Predictions:", correct)
print("Wrong Predictions:", wrong)
# Accuracy
accuracy = correct / len(y_test) * 100
print("Accuracy:", accuracy, "%")
```

16.Implementation Of Agglomerative (Single Linkage) Hierarchical Clustering

X1 ,X2

1,1

1,2

2,1

7,7

7,8

8,7

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.spatial.distance import pdist, squareform
from scipy.cluster.hierarchy import linkage, dendrogram
from sklearn.cluster import AgglomerativeClustering

data = pd.read_csv("data.csv")

D = squareform(pdist(data, metric="euclidean"))
print("Distance matrix:\n", pd.DataFrame(D), "\n")

model = AgglomerativeClustering(n_clusters=2, linkage="single")
data["cluster"] = model.fit_predict(data)
print("Data with assigned clusters:\n", data)
Z = linkage(data.drop(columns="cluster"), method="single")
plt.figure()
dendrogram(Z, labels=[f"P{i}" for i in data.index])
plt.title("Dendrogram (Single Linkage)")
plt.xlabel("Data points")
plt.ylabel("Distance")
plt.show()
```